Production Deployment Considerations

Glovo hourly demand forecasting — 21DM011 Intelligent Data Development, Final Project

María Victoria Suriel · Erika Yazmin Blanco · Elvis Teodoro Casco

This section describes how our team would take the forecasting model from the notebook to a reliable **AWS cloud service** that, every Sunday at 23:59, predicts hourly orders for the coming week and feeds courier staffing. It applies the course concepts (Agile, DevOps, DataOps, MLOps and Cloud) to the concrete solution we built: a *trend-adjusted seasonal-median champion* (SMAPE ~16.3, MASE < 1), deployed as an S3 + Lambda pipeline.

1 Team, roles, interactions and ways of working

We are a team of **three**, organised around the three stages of the pipeline, which mirrors the three self-contained modules we actually delivered (`1_3`, `4_6`, `7_9`). We work **Agile/Scrum** in **two-week sprints**: sprint planning, a short daily stand-up (yesterday / today / blockers), a sprint review with the "business" (the Operations stakeholder role), and a retrospective. The function of **Operations/Sustain** is a role we rotate weekly once the service is live.

Member	Scrum role	Owns	Responsibility
Erika Yazmin Blanco	Developer	`1_3` — Data & EDA	Data ingestion, quality gates, EDA, stationarity/seasonality findings that drive modelling.
María Victoria Suriel	Developer	`4_6` — Modelling	Model competition (naïve, harmonic, SARIMA, XGBoost, seasonal profile), rolling-origin backtest, champion selection.
Elvis Teodoro Casco	Developer + DevOps	`7_9` — Deployment & Decision	Submission, staffing MILP, robustness, **AWS deployment (S3/Lambda/IaC), CI/CD and monitoring**.

Interactions. A teammate took the role of the stakeholder, ensuring that the technical work remained aligned with the operational planning needs. We worked collaboratively to translate the business objective—forecasting courier demand for each hourly slot—into concrete project tasks. Work was organized through a shared backlog, developed in separate Git branches, and reviewed before integration. Tasks were considered complete once they were tested, documented, and agreed upon by the team. Because forecasting projects involve experimentation and iteration, we kept some flexibility in our planning and allowed buffer time for unforeseen challenges.

2 Effort estimates and how they were agreed

During sprint planning, the team estimated the effort required for each major feature using story points, where one point roughly corresponds to one focused day of work. Estimates were discussed collectively until a common view

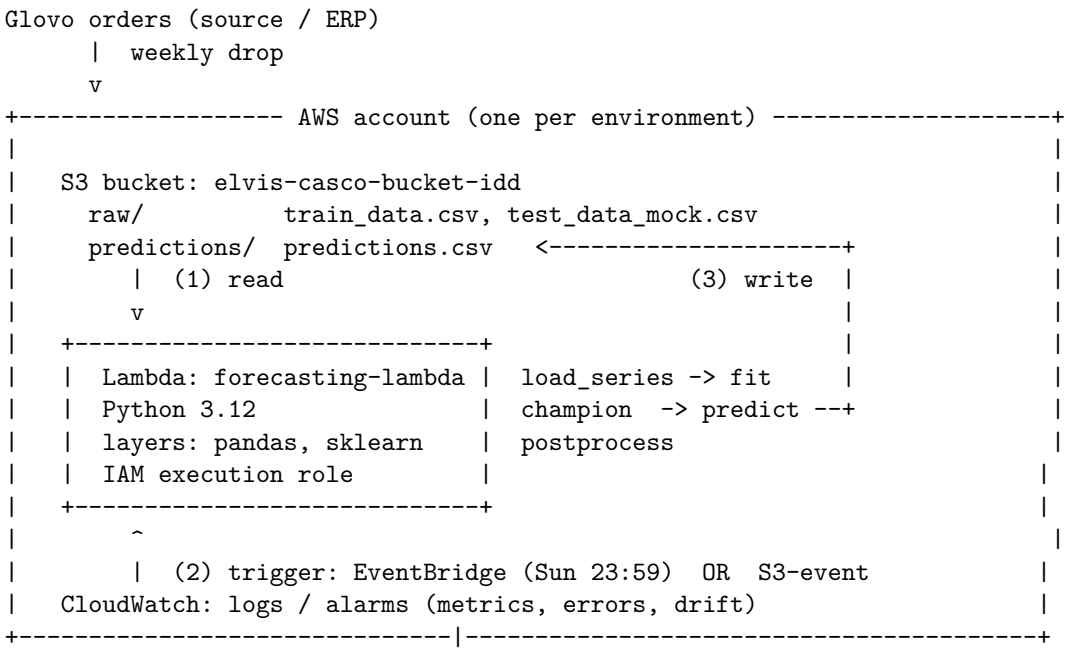
of the effort was reached. We applied a contingency factor to account for uncertainty, integration work, testing, documentation, and unforeseen issues that typically arise in data-science projects

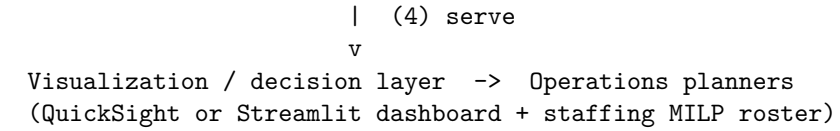
Feature	Owner	Estimated effort (story points)	Key dependencies / remarks
Data ingestion + quality gates (S3 → series)	Erika	3	Schema/row-count/freshness checks
EDA + stationarity/seasonality	Erika	2	Informs feature choice
Model competition + rolling backtest	María	5	Slow (XGBoost/SARIMA refits)
Champion + prediction intervals	María	2	Depends on backtest
Staffing MILP (decision layer)	Elvis	3	Depends on champion
AWS deployment (Lambda + IaC + CI/CD)	Elvis	5	Depends on champion code
Monitoring + alerting + business-continuity fallback	Elvis	3	Cross-cutting

Total estimated effort: approximately 23 story points, corresponding to roughly two sprints for a three-person team. The estimates are intentionally conservative because tasks are only considered complete once they have been implemented, tested, documented, and validated.

3 Architecture of the end-to-end solution

The solution is implemented as a **serverless AWS pipeline**. Data is stored in S3 and processed by AWS Lambda functions running the forecasting code. This design keeps infrastructure costs low, avoids server maintenance, and can scale easily as demand grows or new cities are added.





Components: The solution stores both input data and forecast outputs in **Amazon S3**, including the final predictions.csv deliverable. A **scheduled AWS Lambda function** loads the latest data, runs the forecasting model, and writes the results back to S3. Access between services is controlled through an **IAM role** following the principle of least privilege.

4 Data used and data flows

Data. The dataset consists of hourly order counts for Barcelona from February 2021 to January 2022, yielding approximately 8,500 observations. Because operational planning is performed weekly, the forecasting objective is to predict demand for the next 168 hours. This historical series serves as the basis for all exploratory analysis, model development, and evaluation presented in the project. While the current solution relies solely on historical demand data, the same approach could be extended to incorporate additional demand drivers such as holidays, special events, or weather information.

Flow (Storage → forecasting → visualization). Order data is stored in Amazon S3 and serves as the input to the forecasting pipeline. On a weekly schedule, the forecasting service retrieves the latest data, prepares it for modeling, and generates forecasts for the next 168 hours. The resulting predictions are written back to S3 and made available to downstream consumers. Operations teams can access the forecasts through a reporting or visualization layer, while the staffing optimization module uses them to recommend courier allocations for the coming week. Storing both inputs and outputs in S3 ensures traceability and reproducibility of forecasting runs.

5 Environments and why

To reduce deployment risk and ensure reproducibility, the solution is promoted through four environments: Development, QA, Pre-production, and Production. Each environment serves a different purpose and uses data of increasing realism before changes are released to operational users:

Env	Data	Purpose
Dev	sample/synthetic	Write code, fast iteration; data need not be prod-grade.
QA	prod-like	Validation and automated testing. The goal is to verify that the pipeline executes correctly and that data quality and modeling assumptions are respected
Pre-prod	production-ready	Final validation using production-ready data. Forecasts are reviewed to ensure they are reasonable before deployment to production.
Prod	production	Scheduled weekly run that serves Operations.

This separation reduces the risk of introducing errors into production, allows testing on increasingly realistic data, and ensures that operational forecasts are generated only from validated code.

6 Building and monitoring the ML model

Build Model development is managed through Git/GitHub, with changes developed in separate branches and reviewed before integration. Before a new version is accepted, we verify that the forecasting pipeline executes correctly, that feature generation does not introduce data leakage, and that the rolling-origin backtest reproduces the expected performance. This ensures that modifications improve the solution without breaking existing functionality.

Monitor Before each forecasting run, the input data is checked for completeness, freshness, and consistency. We verify that the expected columns are present, that recent data has been received, and that no obvious anomalies are present. Forecast quality is monitored through metrics such as SMAPE and MASE, which are compared against historical backtest performance. Significant deviations may indicate data issues or changes in demand patterns and trigger further investigation.

Business continuity If critical input data is missing or corrupted, forecasts are not generated automatically. Instead, the issue is flagged for review and the most recent valid forecast can be used as a temporary fallback. This reduces the risk of operational decisions being based on unreliable predictions.

7 How end-users use the solution

The primary users of the solution are Operations and capacity-planning teams. Their role is not to interact with the forecasting model directly, but to use its outputs to support weekly staffing decisions. Each week, the system provides a forecast of hourly demand for the upcoming 168 hours together with a recommended courier allocation plan. Planners can review the forecasts, compare them with operational knowledge or upcoming events, and adjust staffing decisions when necessary. The objective is to support decision-making with consistent, data-driven forecasts while keeping the process simple for non-technical users.

8 Success KPIs

Succes is evaluated at three levels: **forecasting performance**, **business impact** and **operational reliability**:

- **Forecasting performance:** The primary metric is **SMAPE**, which is the evaluation metric used throughout the project. We also report MASE and MSE to assess relative performance against baseline methods and sensitivity to large forecast errors.
- **Business impact:** The forecasts should support effective staffing decisions by reducing under and over-estimation of demand. Better forecasts translate into improved service levels while avoiding unnecessary courier costs.
- **Operational reliability:** The forecasting process should run consistently and produce forecasts on time for weekly planning activities. The solution should also be reproducible and easy to maintain

Target: to improve upon the current manual forecasting process while providing a repeatable and scalable decision-support tool for operations planning.